

Scripts - JobbingTrack

[← Retour à la documentation](#) | [← README principal](#) |  [Navigation](#)

Documentation des scripts d'administration, déploiement et maintenance du projet JobbingTrack.

Vue d'ensemble

Collection complète de scripts shell, Node.js et Python pour automatiser les tâches courantes de développement, déploiement et maintenance.

Catégories de scripts

Scripts de Déploiement

Scripts pour le déploiement et la configuration de l'application en production et développement.

- **Déploiement Docker:** Orchestration conteneurs
- **Configuration environnement:** Variables et secrets
- **Déploiement cloud:** AWS, Azure, GCP
- **Portainer:** Gestion interface graphique

Scripts d'Administration

Scripts pour la gestion et maintenance du système.

Localisation: `scripts/setup/` , `scripts/health/`

- **Installation initiale:** Configuration première installation
- **Vérification santé:** Health checks automatisés
- **Nettoyage:** Logs, cache, fichiers temporaires
- **Backup/Restore:** Sauvegardes automatiques

Scripts Base de Données

Scripts pour la gestion de la base de données PostgreSQL.

Localisation: `scripts/db/`

- **Migrations:** Gestion schéma BDD
- **Seed:** Données de test et développement
- **Backup:** Sauvegardes automatiques
- **Optimisation:** Maintenance et vacuum
- **Monitoring:** Surveillance performances

Scripts de Tests

Scripts pour l'exécution et génération de tests.

Localisation: `scripts/testing/`

- **Tests unitaires:** Exécution par service
- **Tests intégration:** Tests end-to-end
- **Tests performance:** Load testing
- **Génération coverage:** Rapports couverture code
- **Tests CI/CD:** Pipeline automatisé

Scripts Docker

Scripts pour la gestion des conteneurs Docker.

Localisation: `scripts/docker/`

- **Build:** Construction images
- **Cleanup:** Nettoyage images/volumes
- **Logs:** Consultation logs conteneurs
- **Health:** Vérification santé conteneurs
- **Network:** Gestion réseau Docker

Scripts Principaux

Installation et Configuration

```
# Installation complète
./scripts/setup/install.sh

# Configuration environnement
./scripts/setup/configure-env.sh

# Initialisation base de données
./scripts/db/init-database.sh
```

Déploiement

```
# Déploiement développement
./scripts/deployment/deploy-dev.sh

# Déploiement production
./scripts/deployment/deploy-prod.sh
```

```
# Déploiement avec Portainer
./scripts/deployment/portainer-deploy.sh
```

Maintenance

```
# Backup complet
./scripts/db/backup-all.sh

# Nettoyage système
./scripts/maintenance/cleanup.sh

# Optimisation BDD
./scripts/db/optimize.sh
```

Tests

```
# Tous les tests
./scripts/testing/run-all-tests.sh

# Tests par type
./scripts/testing/run-unit-tests.sh
./scripts/testing/run-integration-tests.sh
./scripts/testing/run-e2e-tests.sh
```

Monitoring

```
# Vérification santé système
./scripts/health/check-all.sh

# Métriques système
./scripts/monitoring/collect-metrics.sh

# Alertes
./scripts/monitoring/check-alerts.sh
```



Utilisation via Makefile

Tous ces scripts sont accessibles via le Makefile principal :

```
# Voir toutes les commandes
make help

# Commandes par catégorie
make help-services      # Services Docker
make help-database      # Base de données
make help-tests         # Tests
make help-utils         # Utilitaires

# Exemples
make setup              # Installation initiale
make db-backup          # Backup BDD
make test-all           # Tous les tests
make health             # Vérification santé
```

Sécurité

Bonnes pratiques

✓ Permissions strictes

```
# Scripts sensibles en mode 700
chmod 700 scripts/deployment/*.sh
chmod 700 scripts/db/backup*.sh
```

✓ Variables sensibles

- Utiliser `.env` pour secrets
- Ne jamais commiter secrets
- Utiliser gestionnaire secrets (Vault, AWS Secrets Manager)

✓ Audit logging

- Logger toutes exécutions
- Conserver historique
- Alertes sur échecs

Exemples sécurisés

```
# Utilisation variables environnement
#!/bin/bash
set -euo pipefail
```

```
# Charger .env de manière sécurisée
if [ -f .env ]; then
    export $(cat .env | grep -v '^#' | xargs)
fi

# Vérifier variables requises
: "${POSTGRES_PASSWORD:?Variable POSTGRES_PASSWORD requise}"
```



Dépannage

Script ne démarre pas

```
# Vérifier permissions
ls -l scripts/chemin/vers/script.sh

# Rendre exécutable
chmod +x scripts/chemin/vers/script.sh

# Vérifier shebang
head -n 1 scripts/chemin/vers/script.sh
```

Erreurs d'exécution

```
# Mode debug
bash -x scripts/chemin/vers/script.sh

# Vérifier logs
tail -f logs/scripts/script-name.log
```

Variables manquantes

```
# Vérifier .env
cat .env | grep VARIABLE_NAME

# Afficher toutes variables
env | grep JOBBINGTRACK
```



Ressources

- [Guide de Déploiement](#) - Déploiement complet
- [Scripts Racine](#) - Scripts système complets
- [Guide Makefile](#) - Commandes Make
- [Base de Données](#) - Gestion BDD
- [Tests](#) - Stratégies de tests

Contribution

Ajouter un nouveau script

1. **Créer le fichier** dans le bon dossier
2. **Ajouter shebang** et permissions
3. **Documenter** usage et paramètres
4. **Tester** en local
5. **Ajouter** commande Makefile si nécessaire
6. **Commit** avec message descriptif

Template script bash

```
#!/bin/bash
#
# Nom: mon-script.sh
# Description: Description du script
# Usage: ./mon-script.sh [options]
#
set -euo pipefail

# Configuration
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/../../.." && pwd)"

# Fonctions
function usage() {
    echo "Usage: $0 [options]"
    echo "Options:"
    echo "  -h, --help    Afficher l'aide"
    exit 0
}

function main() {
    echo "Exécution du script..."
    # Logique principale
}

# Parse arguments
while [[ $# -gt 0 ]]; do
    case $1 in
        -h|--help)
```

```
        usage
        ;;
    *)
        echo "Option inconnue: $1"
        usage
        ;;
    esac
    shift
done

# Exécution
main
```

Version: 4.1

Dernière mise à jour: Octobre 2025